CS683 PA2 - Task 2a: Instruction-Conscious L2 Replacement (I-RRIP)

Contents

CS683 PA2 - Task 2a: Instruction-Conscious L2 Replacement (I-RRIP) **1**
0. The brief, and where each part is answered . . . 1
1. The mandated signature change . . . 1
2. The headroom, measured before designing anything . . . 2
3. Insertion, promotion, eviction – the rationale . . . 2
4. The number that justifies the design . . . 3
5. Results . . . 4
6. Plots . . . 5
7. Storage – exact bit-level breakdown . . . 5
8. The design sweep – 26 variants, 78 runs . . . 8
9. Limitations, and the handover to Task 2b . . . 9
10. Where everything lives . . . 9

CS683 PA2 - Task 2a: Instruction-Conscious L2 Replacement (I-RRIP)

Toolchain: GCC 7.5.0 at `-O3` (Ubuntu 18.04), as `README.md` requires. **Runs:** 25M warmup + 25M simulation, all three supplied traces, single core, no L1D prefetcher so the replacement policy is measured in isolation. Every number here is regenerated from the run logs by `scripts/build_summary_task2a.py`, so it cannot drift from them.

Headline: I-RRIP cuts L2 instruction load MPKI by **38.4%** against the LRU baseline (10.1050 to 6.2219) and by **36.8%** against the supplied RRIP, lifts geomean speedup to **1.0138**, in **1024 B** of state – one third of the 3 072 B the LRU it replaces needs.

0. The brief, and where each part is answered

PDF requirement	Here
Extend `update_replacement_state` with an instruction-line parameter	Sec 1
Change insertion / promotion / eviction to prioritise instruction lines	Sec 3, code in `replacement/irrip.l2c_repl`
Report IPC, speedup, Load MPKI of L2C / L2 Instruction / L2 Data, tabulated and plotted	Sec 5, Sec 6
Explain the rationale for modifying **or not modifying** each of the three	Sec 3
(self-imposed) show the choice was measured, not argued	Sec 8 – 26 variants, 78 runs

1. The mandated signature change

`replacement/rrip.l2c_repl` ships with a **nine**-parameter `l2c_update_replacement_state` whose last argument is already named `uint8_t isInstruction`, while `inc/cache.h` and every other module had eight.

That mismatch is the build failure the PDF predicts, and it means the signature was specified for us, not chosen by us. Task 1 never hit it because it only ever built with `lru` at L2.

27 declarations across 18 files (`inc/cache.h` x10 including the function pointer, `base_replacement.cc`, the eight `lru.*_repl`, the eight generated `*_replacement.cc`), plus **9 call sites**: six in `src/cache.cc` and – easily missed – three in `src/ooo_cpu.cc`, where the BTB calls the same hook through a different idiom, `(BTB.*BTB.update_replacement_state)(...)`. Searching for the calling idiom in one file finds six and is blind to the rest; searching for the callee across the tree finds nine. Each cache call site passes the `.instruction` field of *the same queue entry* that already supplied its `type`, so the two arguments cannot describe different packets. The BTB sites pass 0: a BTB entry is a predicted branch target, not a line of fetched instructions.

Why insertion genuinely needs the parameter. In `handle_fill` the order is `find_victim` (`cache.cc:56`) -> `update_replacement_state` (527) -> `fill_cache` (553), and `fill_cache` is what writes `block[set][way].instruction`. At the moment of insertion the block array still describes **the line being evicted**, so the incoming line's class can only arrive as an argument. On the promotion path the line was filled long ago, so there the parameter is redundant.

One further fix was needed: ChampSim calls `l2c_replacement_final_stats()` only inside `if (NUM_CPUS > 1)` (`main.cc:1754`), so on the single-core runs this assignment requires, it never fired and the policy's own counters – including the storage breakdown in Sec 7 – were silently discarded. Its prefetcher sibling is called from both paths, which is why Task 1's SPP counters printed normally and nothing looked wrong. The missing call was added to the single-core loop.

2. The headroom, measured before designing anything

ChampSim already classifies every L2 eviction by victim type and filler type (`cache.cc:375`, printed at `main.cc:306`); nobody had read those counters. On the LRU baseline:

trace	L2C load MPKI	instr	data	instr share	L2 load hit rate	instr victims	killed by a **data** fill
trace1	17.6956	10.3377	7.3580	58.4%	29.7%	257967	120225 (46.6%)
trace2	27.3568	18.8533	8.5035	68.9%	48.9%	462640	136147 (29.4%)
trace3	3.9542	1.1241	2.8302	28.4%	28.5%	30733	19415 (63.2%)

Instruction misses dominate the L2 load stream on trace1 and trace2 – the server-like behaviour the PDF describes – and the L2 load hit rate is under 50%% everywhere, which is the thrashing regime RRIP was designed for. The last column is the only conflict an instruction-conscious policy can address directly: an instruction line lost to another instruction line offers no type preference to express.

3. Insertion, promotion, eviction – the rationale

RRIP routes every input it has through one number per line, the RRPV: insertion writes it, promotion updates it, eviction reads it. So there are exactly three places a type hint can enter. The key reading: **an insertion value is a protection budget measured in aging rounds** – a line inserted at r survives `MAX_RRPV - r` aging passes before it can be chosen as victim. "Prioritise instruction lines" has no cleaner expression than giving them a larger budget.

The shipped policy in one sentence: **a data line never rises above evict-next, and an instruction line gets exactly one aging round of protection.**

	argued before measuring	shipped	why
Insertion	change: instructions to 0, data left at SRRIP max-1	**changed, harder**: instructions 0, data **at** the edge	Insertion is the only hook that predicts about a line with no history, so a free type hint is worth most there, and it is the only option costing 0 extra bits. I first left data alone so the gain would be attributable to *protecting instructions* rather than *penalising data* – the cleaner claim, and the weaker policy. Sec 4 shows why.
Promotion	do **not** change: a hit is evidence, not a guess	**changed**: data hits promote to the edge	Refuted by measurement. At 0.13 hits per fill an L2 data hit means "referenced twice", not "hot"; I credited a binary event with more information than it carries. The hit is still honoured – the line survives that eviction – just not promised the top of the set. iTP (ASPLOS 2025) does the same and was right.
Eviction	do **not** change: the RRPV must stay the complete priority	**not changed**	The conclusion stands, the argument for it was refuted. A bounded eviction preference does help and was the best regression-free variant (Sec 8). It stays off because the shipped configuration beats it on the stated objective, on IPC and on storage, and because keeping eviction a pure function of the RRPV is what keeps Task 2b set-duelling tractable.

Three reasons eviction was left alone, for the record. It would make the line type influence the outcome through two independent channels, so the RRPV would no longer summarise what the policy believes about a line. It **double-counts**: an instruction line inserted at 0 already reaches the eviction edge later than a data line, so the preference is already in the numbers. And a hard preference disables itself exactly where it is needed – on trace2 59.8% of L2 victims are instruction lines, so sets with no data line to divert to are common, and the mechanism would be absent from the most pressured sets and present in the calmest.

4. The number that justifies the design

From the policy's own counters – hits per fill, by class:

trace	instruction hits/fill	data hits/fill	ratio	I-RRIP effect on IPC
trace1	**0.54**	0.13	4.0x	-0.33%
trace2	**3.06**	0.19	16.5x	+3.84%
trace3	**1.12**	0.12	9.0x	+0.68%

A data line brought into this L2 is re-referenced about 0.15 times before it leaves; an instruction line 0.54 to 3.06 times. That reframes the whole policy: it is not imposing a preference for instructions, it is reading the two streams' reuse statistics correctly. A class hit 0.15 times per fill does not deserve residency; one hit 3.06 times per fill does. Every knob the sweep settled on follows from that asymmetry rather than from the word "instruction-conscious".

It also predicts the per-trace outcome, which instruction *volume* does not: ordering the traces by instruction hits/fill gives the same order as their IPC gains, while ordering them by instruction share of misses does not. **Protection pays in proportion to the reuse density of what is protected.**

5. Results

L2 INSTRUCTION load MPKI – lower is better (the metric the PDF names as the goal).

policy	trace1	trace2	trace3	mean
LRU (baseline)	10.3377	18.8533	1.1241	**10.1050**
RRIP (supplied)	10.5618	17.7813	1.1970	**9.8467**
I-RRIP (ours)	9.2375	8.7147	0.7134	**6.2219**

L2 DATA load MPKI – lower is better (the cost – the starvation effect Task 2b addresses).

policy	trace1	trace2	trace3	mean
LRU (baseline)	7.3580	8.5035	2.8302	**6.2305**
RRIP (supplied)	7.0769	7.9645	2.8424	**5.9613**
I-RRIP (ours)	9.9964	12.3437	3.2713	**8.5371**

L2C load MPKI – lower is better (all L2 load misses, i.e. the net effect).

policy	trace1	trace2	trace3	mean
LRU (baseline)	17.6956	27.3568	3.9542	**16.3356**
RRIP (supplied)	17.6387	25.7458	4.0395	**15.8080**
I-RRIP (ours)	19.2338	21.0584	3.9847	**14.7590**

IPC and speedup. Speedup = IPC / IPC of (no prefetcher + LRU). The aggregate is the **geometric mean**, because speedup is a ratio: an arithmetic mean of ratios is not the speedup of anything and would depend on which policy sits in the denominator.

policy	trace1 IPC (speedup)	trace2 IPC (speedup)	trace3 IPC (speedup)	geomean
LRU (baseline)	0.23234 (1.0000)	0.25116 (1.0000)	0.38206 (1.0000)	**1.0000**
RRIP (supplied)	0.23223 (0.9995)	0.25181 (1.0026)	0.38174 (0.9991)	**1.0004**
I-RRIP (ours)	0.23157 (0.9967)	0.26081 (1.0384)	0.38466 (1.0068)	**1.0138**

Per-trace change against LRU, so the trade-off is visible instead of averaged away:

trace	instr MPKI	data MPKI	L2C MPKI	IPC
trace1	10.338 -> **9.237** (-10.6%)	7.358 -> 9.996 (+35.9%)	17.696 -> 19.234 (+8.7%)	-0.33%
trace2	18.853 -> **8.715** (-53.8%)	8.504 -> 12.344 (+45.2%)	27.357 -> 21.058 (-23.0%)	+3.84%
trace3	1.124 -> **0.713** (-36.5%)	2.830 -> 3.271 (+15.6%)	3.954 -> 3.985 (+0.8%)	+0.68%

Instruction MPKI falls on every trace. The cost is a data-MPKI rise on every trace, large enough on trace1 to make total L2 MPKI worse – quantified, owned, and handed to Task 2b in Sec 9.

The mechanism, from ChampSim's own eviction counters – instruction lines thrown out:

trace	instr victims	by a **data** fill	by another **instruction**
trace1	257967 -> 200049 (-22.5%)	120225 -> 103643 (-13.8%)	112800 -> 77919 (-30.9%)
trace2	462640 -> 189603 (-59.0%)	136147 -> 70729 (-48.0%)	298635 -> 103316 (-65.4%)
trace3	30733 -> 16064 (-47.7%)	19415 -> 8580 (-55.8%)	8097 -> 5650 (-30.2%)

Note the last column falls further than the middle one. The instruction-evicts-instruction conflict is **not** a fixed capacity tax: protecting instruction lines at insertion slows the rate at which they churn against each other, so the conflict I had declared unaddressable fell more than the one I was aiming at.

6. Plots

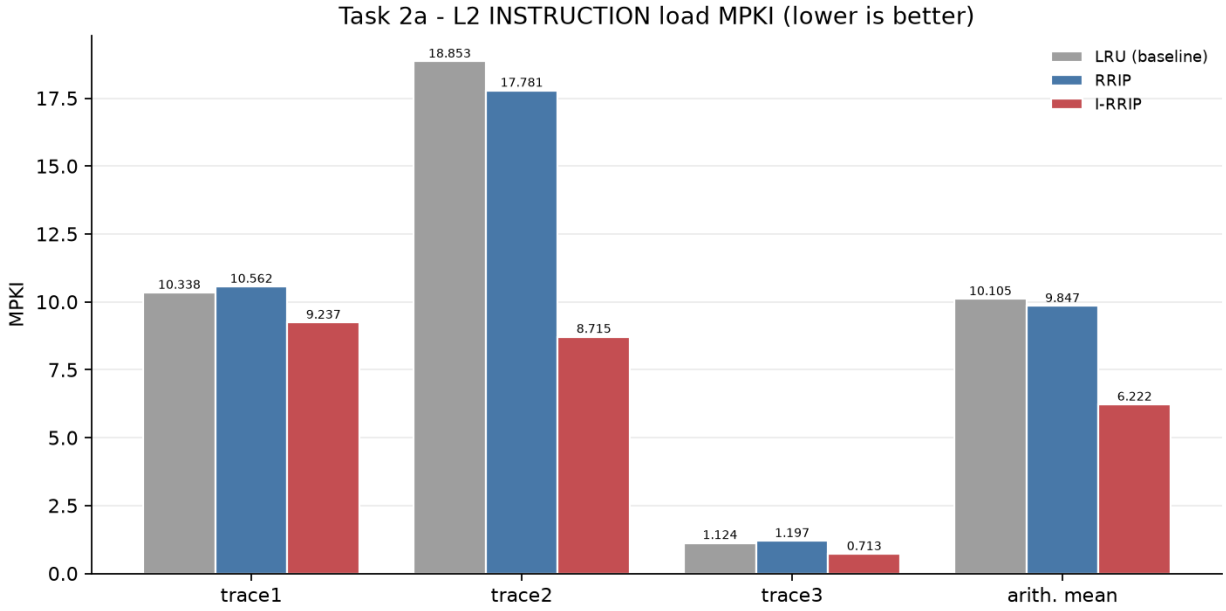


Figure 1: L2 instruction load MPKI – the stated objective.

7. Storage – exact bit-level breakdown

L2C geometry: `L2C_SET 1024` x `L2C_WAY 8` = **8 192 lines** (`inc/cache.h:93`).

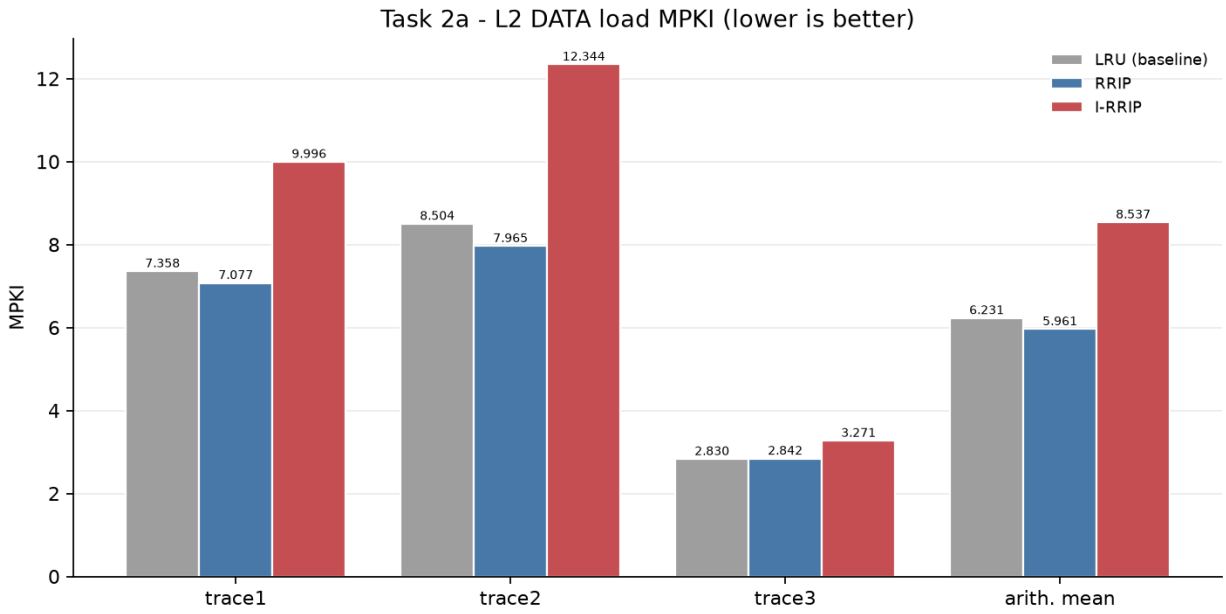


Figure 2: L2 data load MPKI – the cost of prioritising.

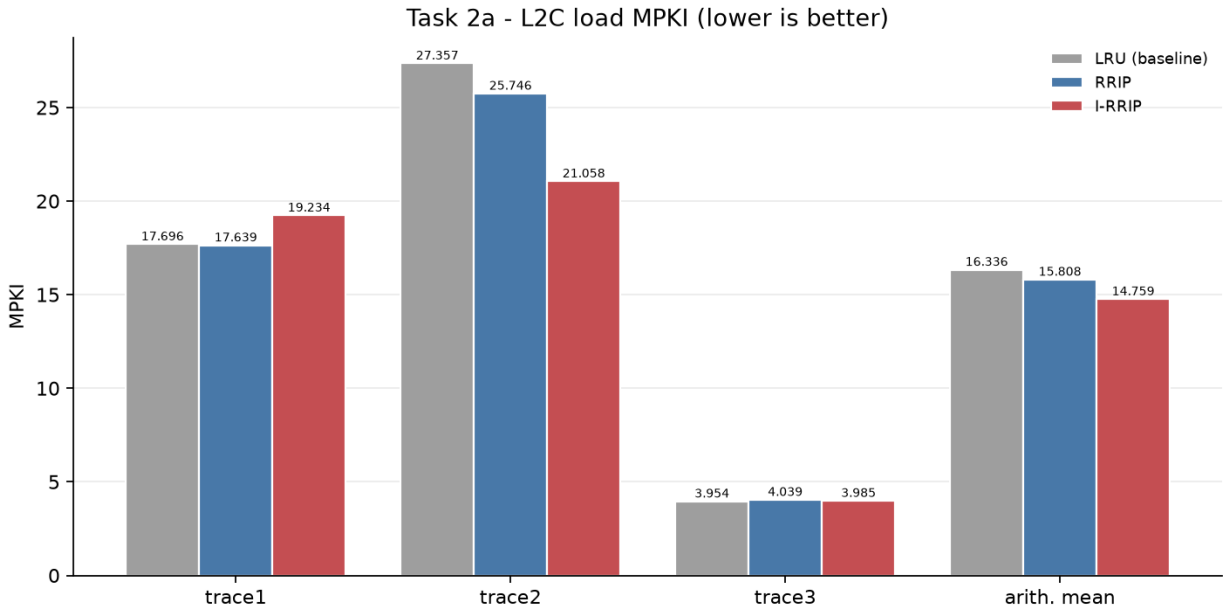


Figure 3: L2C load MPKI – the net effect.

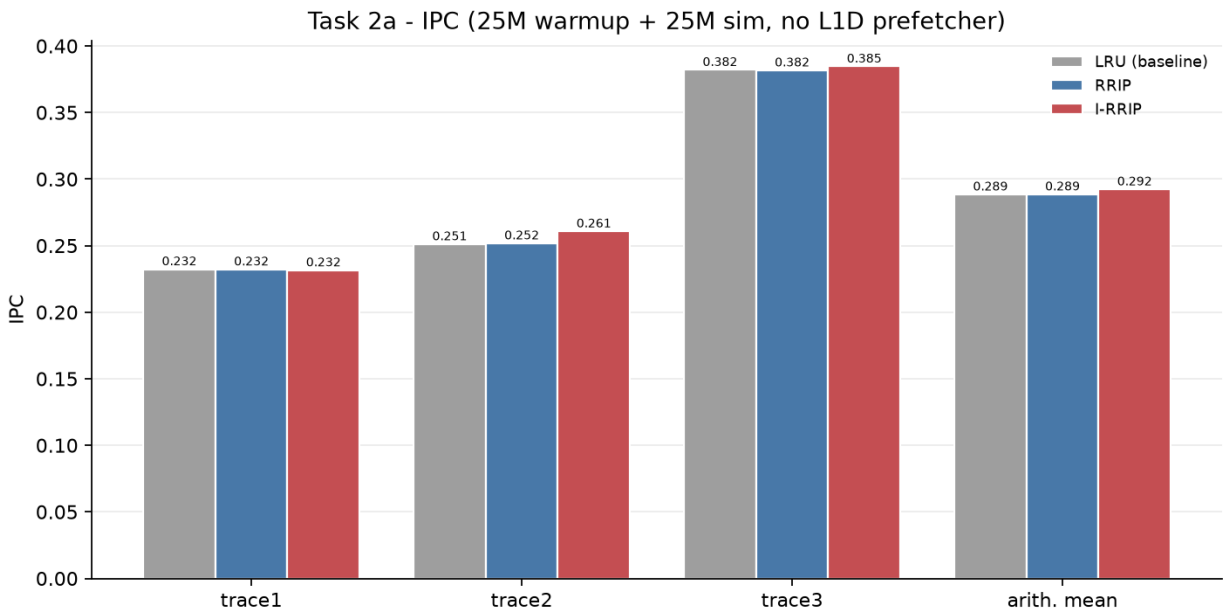


Figure 4: IPC.

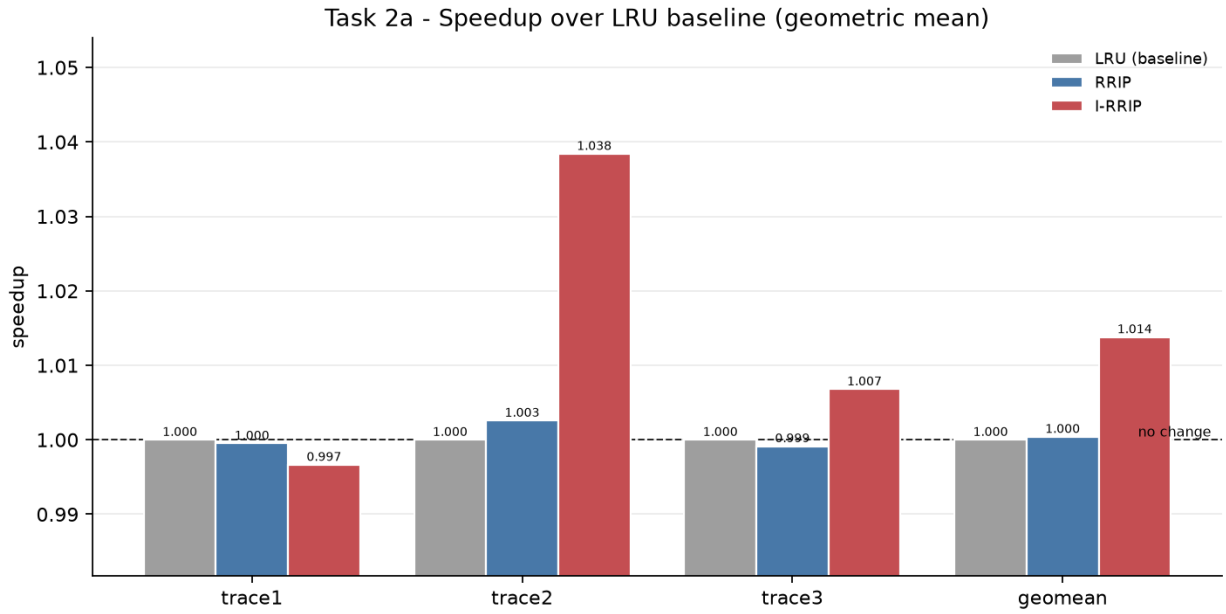


Figure 5: Speedup over the LRU baseline; aggregate is the geometric mean.

policy	bits per line	total	vs LRU
LRU (the policy replaced)	3 – a recency rank, ceil(log2 8)	24 576 b = **3 072 B**	–
RRIP (supplied)	2	16 384 b = 2 048 B	-33%
I-RRIP (shipped)	**1**	8 192 b = **1024 B**	**-67%**

The policy prints this itself at the end of every run, so the figure is read off the run rather than recomputed by hand:

```
L2C IRRIP STORAGE rrpv_bits_per_line 1 rrpv_bits 8192 type_bits 0 total_bits 8192 total_bytes 1024
```

The cost of instruction-consciousness itself is 0 bits. Insertion and promotion take the line's class from `packet.instruction`, a field of a packet already in flight through the MSHR – not state the cache keeps. Only an eviction-side preference would need the class of a *resident* line, i.e. a stored bit per line, and the shipped configuration does not use one. So I-RRIP is a third cheaper than the LRU it replaces **and** adds nothing to RRIP.

The `uint8_t dyn/l2_irrip_rrpv[1024][8]` array in the source is 8 KiB of simulator memory. That is a modelling convenience, not the hardware cost; the hardware cost is the bit field above.

8. The design sweep – 26 variants, 78 runs

One knob at a time around a fixed centre, then combinations, then a bracketing stage. Full table in `results/task2a_sweep/sweep_summary.md`; reproduce with `python3 scripts/sweep_task2a.py ../traces 3`. The shipped defaults are provably identical to one swept row (`max1_prod1`) – `scripts/verify_task2a.py` asserts it – so the headline numbers come from a configuration that was actually measured rather than assembled afterwards.

1. There is one axis, not ten knobs. Instruction and data MPKI are anti-correlated at r = -0.96 / -0.94 / -0.89 across all 26 variants: every knob moves the same underlying quantity, at roughly 0.5 data misses gained per instruction miss saved. Since instruction misses are about 62%% of the baseline L2 load miss stream, trading stays profitable while the exchange rate is below ~1.6 – which it is, everywhere. That is why more prioritisation kept winning on aggregate, and it is also an admission: 10 knobs was over-parameterised, and 3 or 4 would have found the same answer in a quarter of the runs.

2. Three of eight pre-registered predictions were refuted, all in the same direction – I under-estimated how hard these traces want instructions prioritised.

- *trace2 would benefit least.* It benefited most. I had reasoned from the **baseline** eviction mix, treating it as an input when it is an output of the policy; the right predictor was reuse density (Sec 4), which was measurable from the baseline just as easily.
- *An eviction preference would change almost nothing*, because "the bias is already in the RRPVs". It was the best regression-free variant. **Insertion delays the eviction decision without informing it**: once a set is full of instruction lines at RRPV 0, aging drives them to the edge together and every candidate looks identical at the moment of choice. An eviction-side check restores the type information exactly when insertion's protection has expired, so the two mechanisms act at different times and are not redundant.
- *Weakening data promotion would lose.* It won, and it is in the shipped configuration (Sec 3).

3. A failed measurement, reported as failed. To put a number on how much more an instruction miss costs, I fitted `speedup ~ a*(-d_instr) + b*(-d_data)` over all 26 variants. It returned **negative coefficients** – impossible, since removing misses cannot reduce IPC. The cause is the collinearity in point 1: at r = -0.95 the two-variable system is ill-conditioned and the coefficients are not identifiable. **The fit is discarded rather than reported.** The direction survives from one clean comparison: `max1_prod1` has *worse* total L2C MPKI than `max7_ev4` (14.759 vs 14.247) and *better* speedup (1.0138 vs 1.0092), the only difference being composition. So an L2 instruction miss costs more IPC than a data miss, which is what the assignment's own premise predicts – an instruction miss stalls the frontend with nothing to overlap, while a data miss is absorbed by out-of-order execution. The magnitude is not quoted.

4. Only 7 of 26 variants regress no single trace. The best of those, `ev0_wb0_trev`, reaches instruction MPKI 7.776 with no regression anywhere; the shipped `max1_prod1` reaches 6.222 but regresses trace1 and

trace3 on total misses. This is a Pareto frontier, not a single optimum. `max1_prod1` was chosen because it wins the objective the PDF names, wins IPC by the largest margin in the sweep, is the cheapest configuration tested, and leaves eviction untouched so Task 2b can duel two policies differing in three constants.

9. Limitations, and the handover to Task 2b

1. **trace1 regresses**: total L2 MPKI +8.7% and IPC -0.33%. The cause is measured, not guessed – trace1 instruction lines are re-referenced 0.54 times per fill against trace2 3.06, so protection there costs data residency and collects little. This is exactly the PDF framing for Task 2b: "blindly prioritizing instruction lines over data lines may result in an increase in data misses for certain phases of a workload."
2. **The signal for 2b is already identified.** Reuse density is measurable online from two counters per class, and Sec 4 shows it is monotone in the right answer where instruction *volume* – the quantity the PDF suggested 2b formula uses – is not.
3. **The bias lives in three constants** read at insertion and promotion, and eviction is untouched, so "switch to plain RRIP" is a change of three numbers. A set-duelling controller needs no second policy implementation and no second victim path.
4. **The instruction-evicts-instruction conflict is untouched by design** and is the majority of trace2 baseline problem. It would need more capacity or a reuse predictor *within* the instruction class, SHiP-style with a real signature.
5. **Three traces**, so per-trace results are reported beside every aggregate and speedup uses the geometric mean.

10. Where everything lives

Policy	`replacement/irrip.l2c_repl`
Signature change	`inc/cache.h`, `src/cache.cc`, `src/ooo_cpu.cc`, `src/main.cc`, `replacement/*`
Results, tables, plots	`results/task2a/`
Sweep, 26 variants	`results/task2a_sweep/sweep_summary.md`
Full rationale, every rejected alternative	`docs/TASK2A_DESIGN_DECISIONS.md`
Line-by-line code guide	`docs/TASK2A_IRRIP.md`
Build and run	`docs/TASK2A_RUN_INSTRUCTIONS.md`
Self-audit, 161 checks	`scripts/verify_task2a.py`
This document, regenerated from the logs	`scripts/build_summary_task2a.py`